

Agenda

① What did LLM's change??—

② How LLM's work end to end

- Tokenization
- Vectorization
- Positional Encoding
- Attention
- Feed forward NN
- Output's

2019

Imagine you've written some code. You run it. It crashes with an error.

What did you do two or three years ago?

You'd copy the **error message** — just the error — and paste it into Google.

Now think about this: you never pasted the *code* itself. Why not?

Google was a search engine. All it could do was find pages that contained the same words as your query.

That's all Google ever did. **Keyword matching**. It didn't read your error. It didn't understand your error. It just hoped someone else had typed those same characters before. —

2026

You hit the same kind of bug.

What do you do?

You paste the whole code. Sometimes you don't even paste the error — you just paste the code and say "*find the bug.*" And it does.

The machine isn't searching anymore. It's *reading*. It's looking at your code and **understanding** what it's supposed to do,

So what changed between 2019 and 2026?

We solved a problem in machine learning that had been open for decades: we figured out how to make machines understand language instead of just matching it.

the thing that turned pattern-matching machines into reading machines — is called attention.

Attention is what lets a model look at one word in your code and instantly know which other words around it matter for understanding it.

Before attention: machines matched characters. After attention: machines read meaning.

```
total = 0
for price in prices:
    total = price
print(total)
```

— How do you understand the meaning of total???

What Actually Happens Inside The Model - How LLM's work

```
total = 0
for price in prices:
    total = price
print(total)
```

"Why does this print only the last price instead of the sum?"

The model takes this whole thing — code plus your question — and runs it through five steps. Each step is a transformation. By the end, it produces an answer.

Step 1: Tokenise-> Turn text into numbered pieces

The model can't read text. It can only do math on numbers. So before anything else, your input gets chopped into **tokens** — small pieces of text from a fixed vocabulary it learned during training.

"total" → 14336

" =" → 284

" 0" → 657

"\n" → 198

"for" → 1640

" price" → 4536

" prices" → 6634

"print" → 1762

...

The model still understands nothing. They're just serial numbers.

Step 2: Embedding — Turning IDs Into Meaning Vectors

Here's where things get interesting. The serial number 14336 (total) gets looked up in a giant table and *replaced with a profile* — a long list of numbers that describe what total means called (vectors)

"total" -> [0.9 0.9 0.8 0.85 0.7]

"price" -> [0.4 0.1 0.7 0.0 0.6]

"prices" -> [0.4 0.1 0.7 0.9 0.6]

"for" -> [0.0 0.5 0.95 0.0 0.1]

"=" -> [0.0 0.7 0.95 0.0 0.4]

"print" -> [0.0 0.8 0.95 0.0 0.0]

A few things to notice:

total – an English noun for "the whole amount,"
commonly reused as a code variable name

- noun-ness: 0.9
- aggregate / sum. 0.9
- represents-a-quantity: 0.8
- typical-variable-name: 0.85
- accumulator-pattern assoc: 0.7 (appears next to "+= x" all the time)

price – a money word, a common business-code variable

- noun-ness: 0.9 ← shared with total: both are nouns
- money / commerce concept: 0.95
- single-item value: 0.85
- countable-thing: 0.8
- typical-variable-name: 0.7

prices – same neighborhood as price, but plural

- noun-ness: 0.9 ← same
- money / commerce concept: 0.95 ← same as price
- collection / plural: 0.95 ← THE distinctive trait
- iterable-suggestive: 0.85 ← "you can loop over me"
- typical-variable-name: 0.7

for – a Python keyword, a loop construct

- Python keyword: 0.95
- loop / iteration: 0.95
- control-flow construct: 0.9
- expects-an-identifier-next: 0.85 ← it knows "for X in ..." structure
- function-word (not content): 0.9 ← like "the" or "and"

= — the assignment operator that overwrites

- *assignment operation:* 0.95
- *mutation / state-change:* 0.9
- *single-character operator:* 0.95
- *overwrites:* 0.85 ← replaces the old value entirely
- *accumulator-pattern:* 0.1 ← does NOT accumulate

print

- *function / callable:* 0.95
- *output / IO:* 0.95
- *side-effect:* 0.9
- *debugging-tool-associated:* 0.7 ← people use print() to debug
- *common-across-languages:* 0.9 ← print() exists in nearly every language

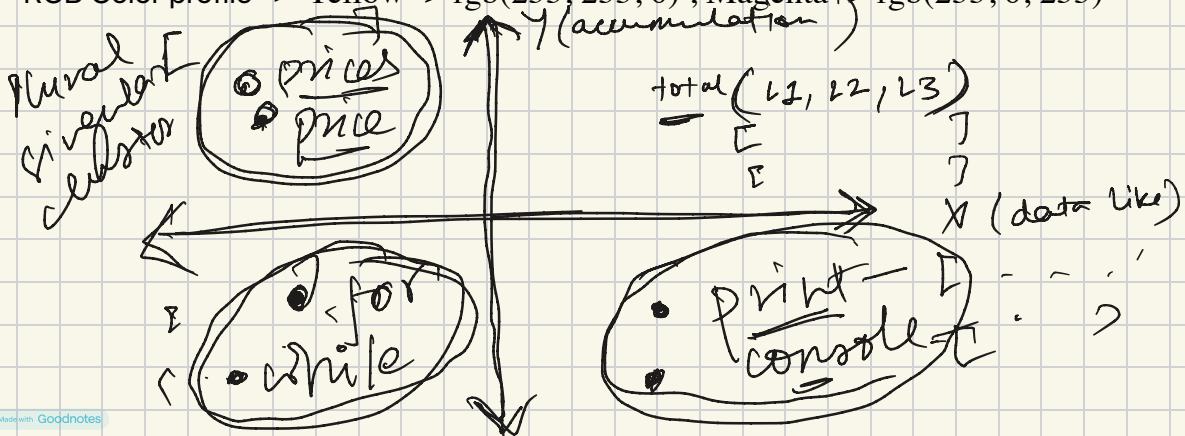
This is the moment text becomes math. Now you can do things like:

`vector("king") - vector("man") + vector("woman") ≈ vector("queen")`

Now step back and look at what just happened.

We have converted them into mathematical expressions but those mathematical expressions have the meaning of the word

RGB Color profile -> Yellow -> `rgb(255, 255, 0)`, Magenta -> `rgb(255, 0, 255)`



Positional Encoding

`total = price`

`price = total`

After tokenization and embedding, both of these produce the **exact same set of vectors** — one for `total`, one for `=`, one for `price`. The vectors are identical because the words are identical. But the meaning is completely opposite.

Before the vectors enter the attention layers, the model adds a small "position signal" to each one that says "I am the 1st word," "I am the 2nd word," "I am the 3rd word," etc.

Position 1: "total" → embedding + "I'm at position 1"

Position 2: " =" → embedding + "I'm at position 2"

Position 3: " 0" → embedding + "I'm at position 3"

Position 4: "\n" → embedding + "I'm at position 4"

Position 5: "for" → embedding + "I'm at position 5"

Position 6: " price" → embedding + "I'm at position 6"

...

Position 11: "total" → embedding + "I'm at position 11"

Position 12: " =" → embedding + "I'm at position 12"

Position 13: " price" → embedding + "I'm at position 13"

...

Now look at what happened to the three `total`s:

- `total` at position 1 has its embedding **plus** a position-1 signal
- `total` at position 11 has the same embedding **plus** a position-11 signal
- `total` at position 15 (inside `print`) has the same embedding **plus** a position-15 signal

"The dog bit the man" vs "The man bit the dog"

Exact same five words. Exact same embeddings. Completely opposite meaning.

Without positional encoding, the model sees both sentences as identical — just a bag containing the, dog, bit, the, man. It has no idea who bit whom.

With positional encoding:

In sentence 1: dog is at position 2 (before bit) → it's the one doing the biting

In sentence 2: dog is at position 5 (after bit) → it's the one getting bitten

Same word. Different position. Opposite role.

Step 3 Attention: Give every word the context of its neighbors

total = 0 ← total on line 1
for price in prices:
 total = price ← total on line 3
 print(**total**) ← total on line 4

The model at this point understands: *what each word generally is but not what each word is specifically doing in this code.*

For every word, attention asks: "what's happening around me?" — and then updates that word's meaning to reflect the answer.

total on line 1 (`total = 0`)

Before attention: "I'm total. I generally mean a sum."

Attention looks around and sees: there's a = right next to me, and a 0 right after that.

After attention: "I'm total, and I'm being set to zero. I'm being initialized"

total on line 3 (`total = price`)

Before attention: "I'm the word total. I generally mean a sum." (Same as above — identical.)

Attention looks around and sees:

there's a = next to me, and price after that. There's a for loop above me. I'm *inside* the loop body.

After attention: "I'm total, and I'm being **overwritten with price** on every iteration of the loop. I'm not accumulating anything. I'm just getting replaced."

total on line 4 (`print(total)`)

Before attention: "I'm the word **total**. I generally mean a sum." (*Still identical.*)

Attention looks around and sees:

I'm inside `print()`.

There's a **total** on line 1 that was set to 0.

There's a **total** on line 3 that gets overwritten with **price** each loop iteration.

After attention: "I'm **total**, and I'm being **printed**. My value is whatever the last loop iteration left me with — which is the last **price**, not a sum."

= on line 3 (`total = price`)

Before attention: "I'm an assignment operator. I set something equal to something."

Attention looks around and sees: **total** is on my left. **price** is on my right.

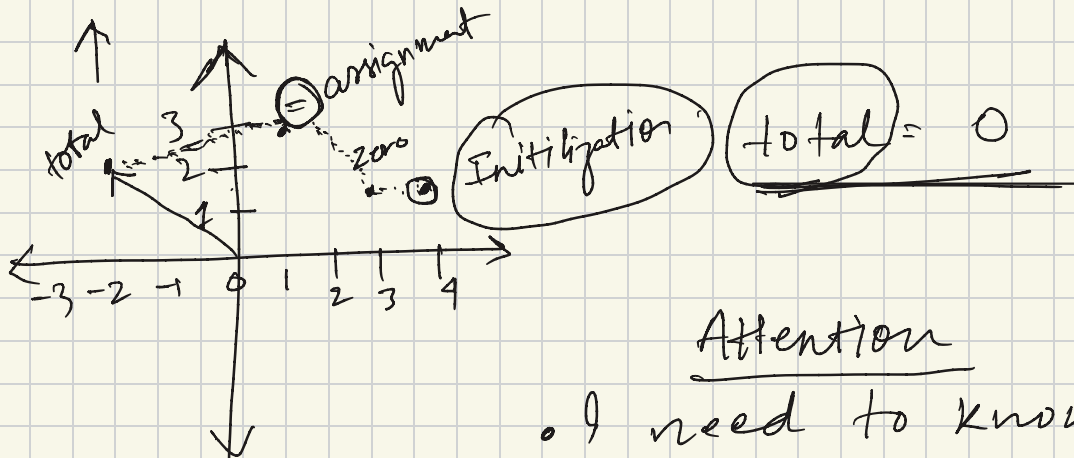
There's a **for** loop above me. This is happening repeatedly.

After attention: "I'm assigning **price** into **total**, **over and over**, replacing the old value each time. I'm not **+=**. I'm **=**."

Before attention: every word knows what it means in a dictionary.

After attention: every word knows **what it's doing in this specific code**.

The three **total**s started identical. After attention, they mean three different things: "initialized zero," "overwritten with **price** inside a loop," and "printed at the end."



$$\text{total} = [-2, 2] \checkmark$$

$$\text{assignment} = [1, 3] \checkmark$$

$$\text{zero} = [3, 1] \checkmark$$

$$\text{Initialization}(4, 1)$$

$$\text{nudge} = [-2, 2] * [1, 3]$$

Attention

- I need to know what an i doing??
- Absorb meaning from surrounding context

$$v(\text{total}) = \text{nudge from } (v\text{-assignment})$$

$$+ \text{nudge from } (v = 0)$$

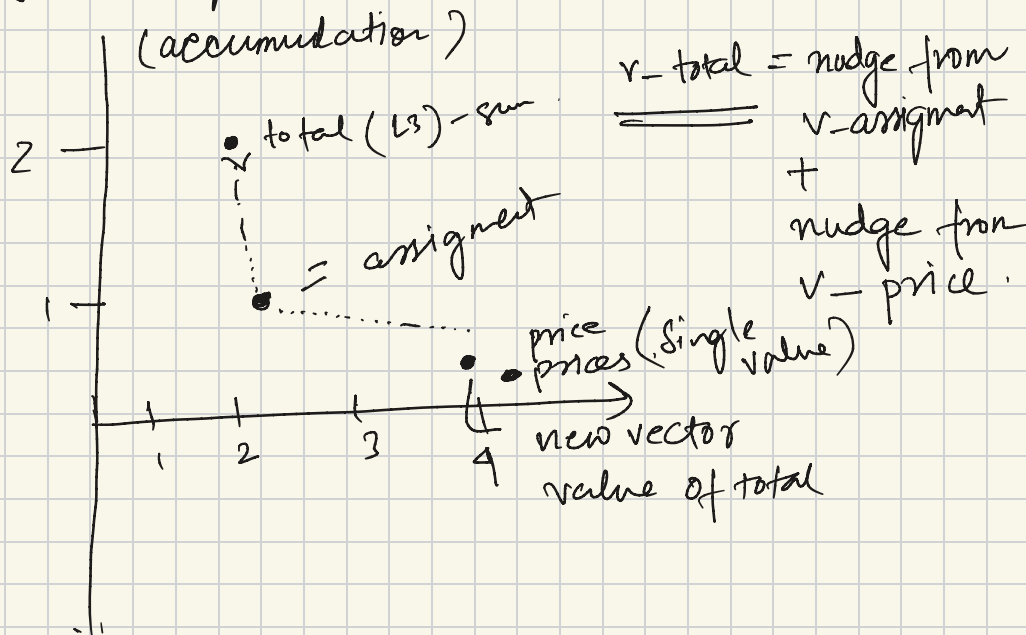
$$\rightarrow v(\text{initialization})$$

total = 0

for price in prices

(total = price ← what happens here
print total)

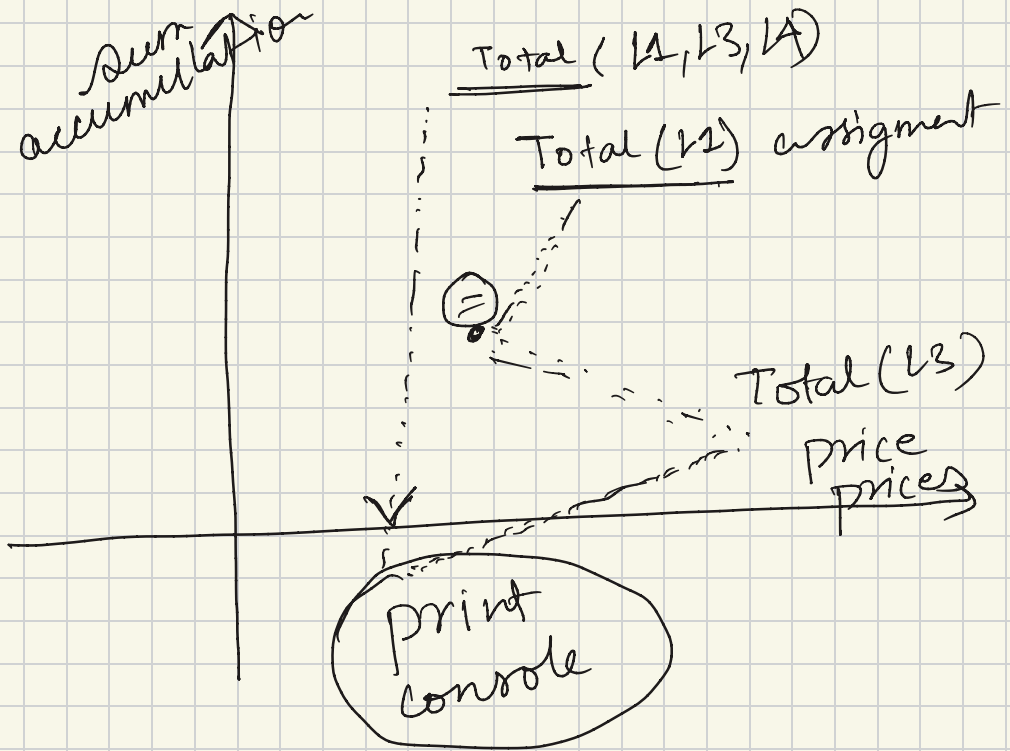
Strongest things around are =, total



- total from line (1) will also have impact but minimal (Matrix rule)
- loop will also have an impact but minimal.
- In the accumulation axis it has moved down. Single value overwritten.

What happens to `total` on line 4

It also looks at the **other two `total`s**, which have already been moved by attention.



`total(L4)` no longer means "a running total being printed."
It now means "a value being printed that isn't actually accumulating anything — just the last price assigned."

Question 1: Where does the word `total`, by its name, want to live?

Question 2: Where did `total(L4)` actually end up after attention?

That mismatch is the bug. The geometry now contains a contradiction: the word is named like an accumulator but it behaves like an overwriter.

Input to attention:

Output of attention:

total(L1) [2,3] **original_v** → total(L1) [5,6] [**original_v** + **absorbed context**]

=	[original]	→	=	[original + absorbed context]
0	[original]	→	0	[original + absorbed context]
for	[original]	→	for	[original + absorbed context]
price	[original]	→	price	[original + absorbed context]
in	[original]	→	in	[original + absorbed context]
prices	[original]	→	prices	[original + absorbed context]
total(L3)	[original]	→	total(L3)	[original + absorbed context]
=	[original]	→	=	[original + absorbed context]
price	[original]	→	price	[original + absorbed context]
print	[original]	→	print	[original + absorbed context]
total(L4)	[original]	→	total(L4)	[original + absorbed context]

...